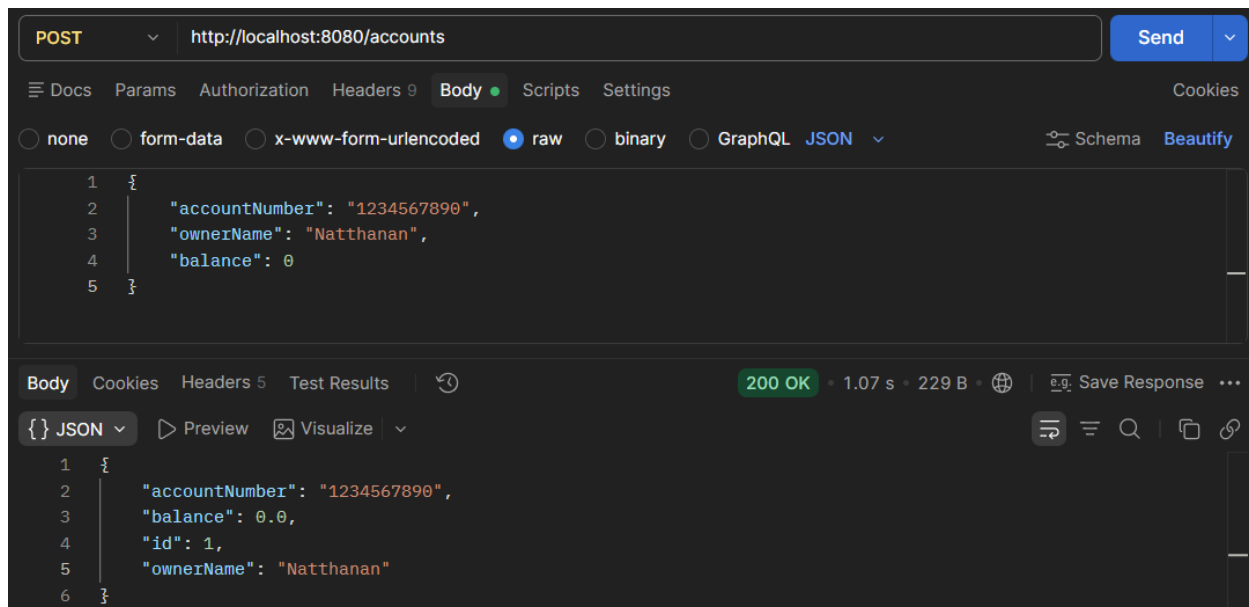
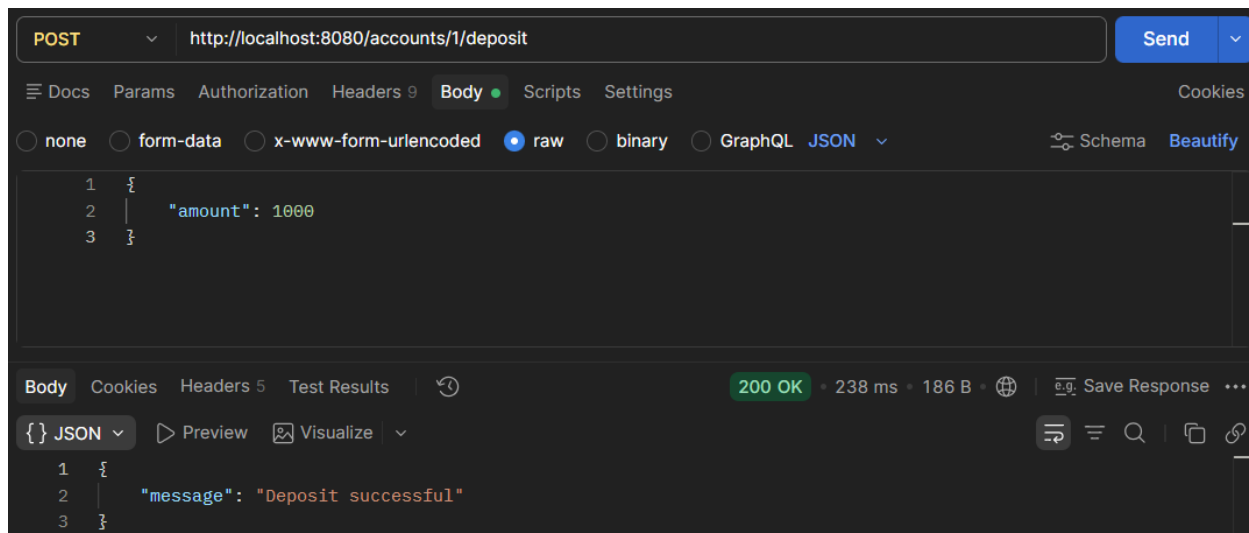


11.1 สร้าง Account



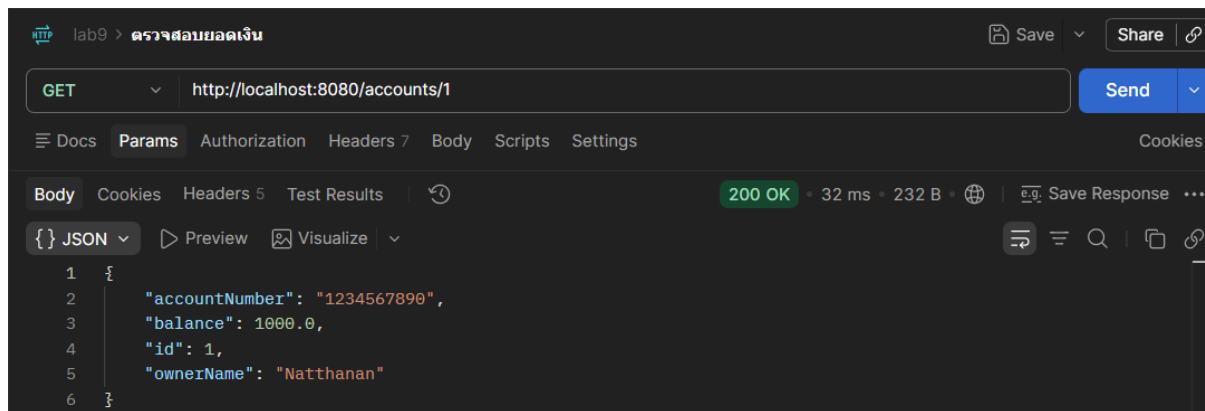
ส่ง POST /accounts พร้อมข้อมูล Account และได้รับสถานะ 200 OK โดยระบบสร้าง Account สำเร็จ และกำหนด id = 1 พร้อมยอดเงินเริ่มต้น 0.0 บาท

11.2 ฝากเงิน



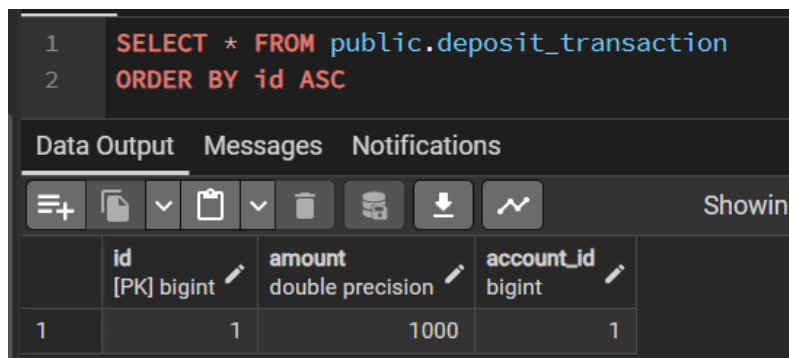
ส่ง POST /accounts/1/deposit ด้วยจำนวนเงิน 1000 บาท ระบบตอบกลับ 200 OK และแสดงข้อความ Deposit successful แสดงว่าการฝากเงินทำงานสำเร็จ

11.3 ตรวจสอบยอดเงิน



ส่ง GET /accounts/1 พบว่า Account มี balance = 1000.0 บาท แสดงว่ายอดเงินใน Account ถูกเพิ่มจาก 0 เป็น 1000 บาทหลังจากฝากเงินสำเร็จ

11.4 ตรวจสอบประวัติการฝากเงิน



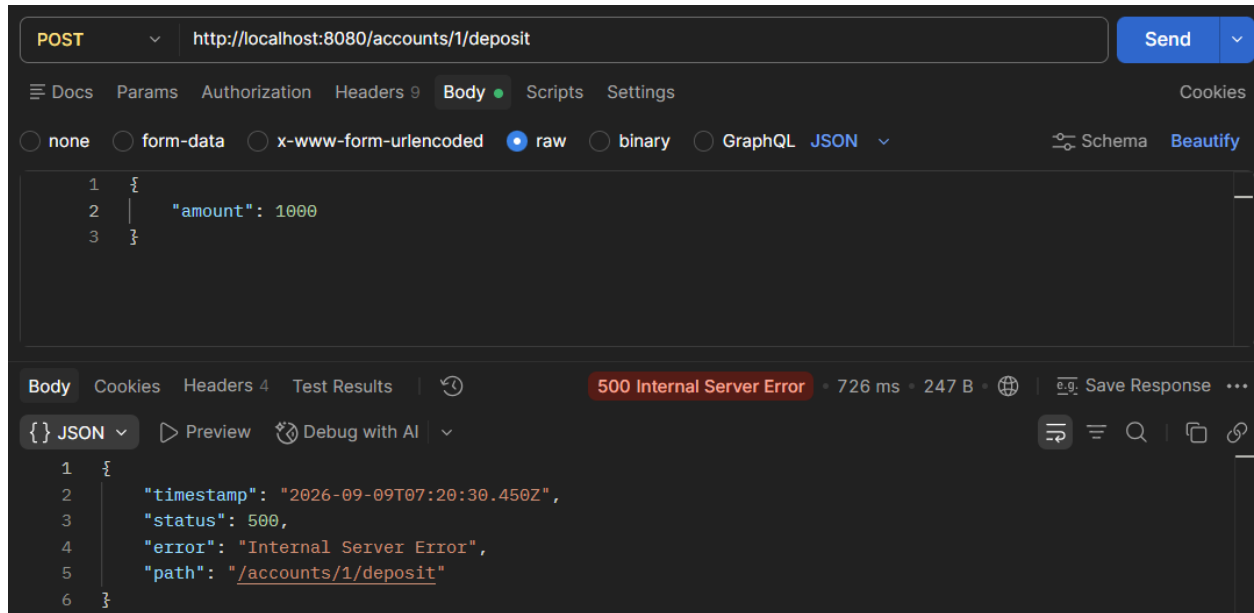
ตรวจสอบตาราง deposit_transaction ใน PostgreSQL พบข้อมูลจำนวน 1 รายการ แสดงว่าการฝากเงินได้บันทึกประวัติลง Database สำเร็จ

12. เพิ่มโค้ดที่ DepositService

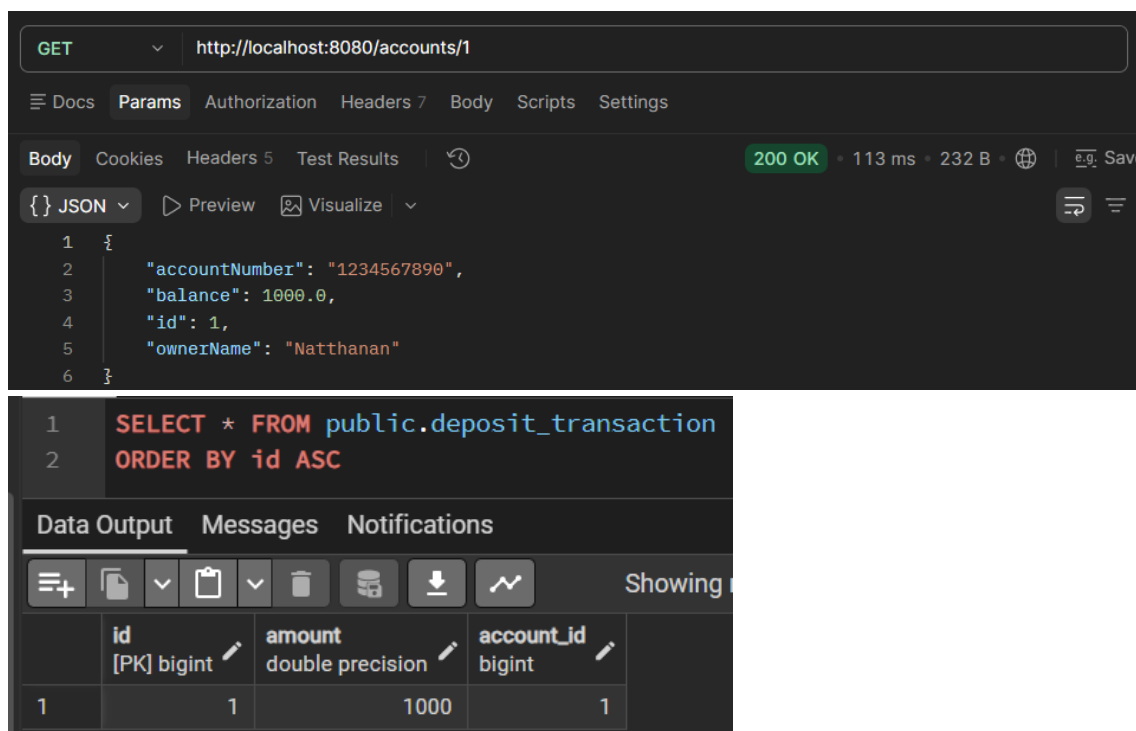
```
throw new RuntimeException(message: "Test Rollback");
```

หลังจากบันทึก Account และ DepositTransaction แล้ว จากนั้นส่ง Request ฝากเงินอีกครั้ง ระบบตอบกลับ 500 Internal Server Error ซึ่งเป็นผลจาก Exception ที่ตั้งใจสร้างขึ้น

12.2 ยิง Request เข้าด้วย Postman



12.3 ตรวจสอบว่าข้อมูลถูกย้อนกลับหรือไม่



เมื่อใช้ @Transactional แล้วตรวจสอบข้อมูลอีกครั้ง พบว่า

ยอดเงิน Account ยังคงเป็น 1000 บาท และตาราง deposit_transaction ยังคงมีเพียง 1 รายการ
รายการฝากเงินครั้งที่สองจำนวน 1000 บาท ไม่ถูกบันทึก

12.4 สรุปผล

เมื่อเกิด Error ระบบทำการ ROLLBACK ข้อมูลทั้งหมดที่เกิดขึ้นภายใน Transaction จึงไม่ถูกบันทึกลง Database นั่นคือ แม้โปรแกรมจะทำการเพิ่มยอดเงินและบันทึกประวัติไปก่อนเกิด Error แต่เมื่อเกิด Exception ระบบจะย้อนกลับข้อมูลทั้งหมดของ Transaction นั้น ทำให้ Database กลับไปอยู่ในสถานะเดิม

13. ทดลองโดยเอา @Transactional ออก

```
//@Transactional
public void deposit(Long accountId, Double amount) {

    // 1. ค้นหา Account
    Account account = accountRepository.findById(accountId)
        .orElseThrow(() -> new RuntimeException(message: "Account not found"));

    // 2. เพิ่มเงินเข้า balance
    account.setBalance(account.getBalance() + amount);
    accountRepository.save(account);

    // 3. สร้างประวัติการฝากเงิน
    DepositTransaction transaction = new DepositTransaction(amount, account);

    depositRepository.save(transaction);

    // ใช้ Rollback ข้อ 12
    throw new RuntimeException(message: "Test Rollback");
}
```

ตอนที่ @Transactional

POST http://localhost:8080/accounts/1/deposit Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "amount": 1000
3 }
```

Body Cookies Headers 4 Test Results 500 Internal Server Error 726 ms 247 B Save Response

{ JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-09-09T07:20:30.450Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

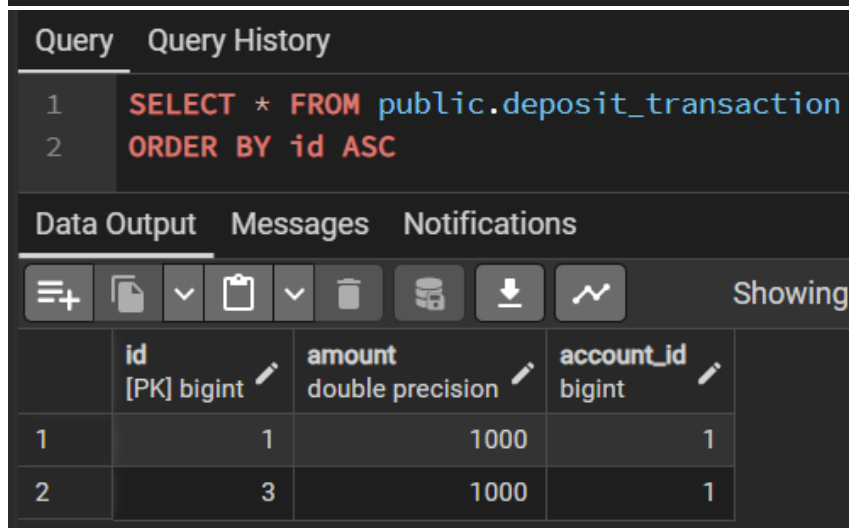
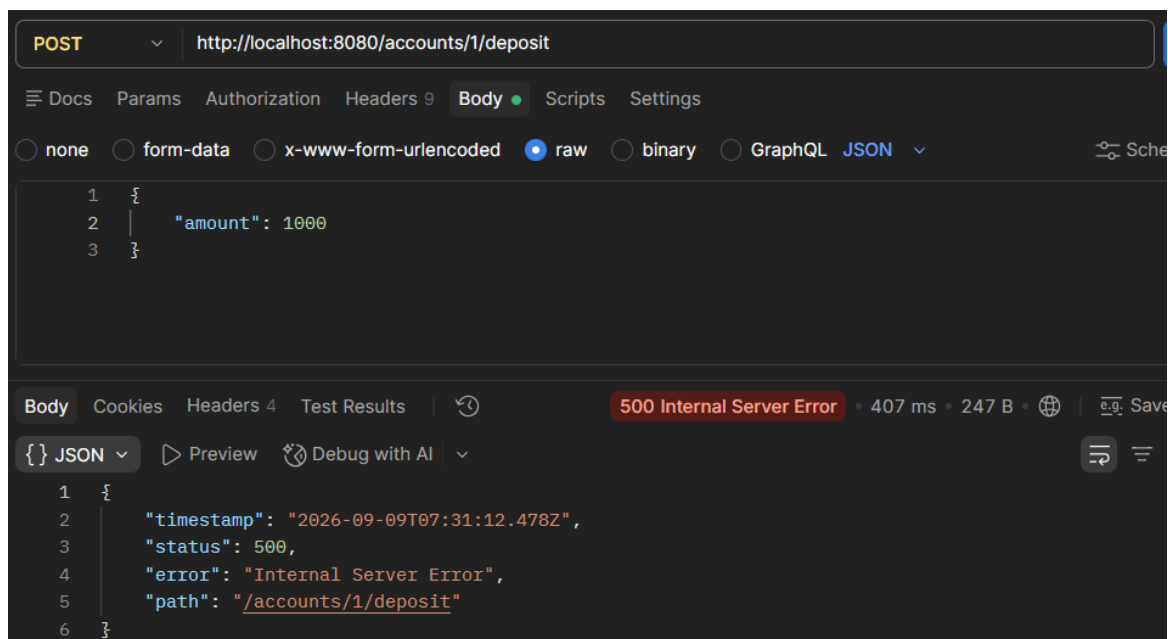
```
1 SELECT * FROM public.deposit_transaction
2 ORDER BY id ASC
```

Data Output Messages Notifications

Showing 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

ตอนที่ไม่มี @Transactional



ผลที่พบเมื่อไม่มี @Transactional

Request ฝากเงินยังคงตอบกลับ 500 Internal Server Error เนื่องจากเกิด Exception แต่ข้อมูลที่ทำไป ก่อนเกิด Error ยังคงถูกบันทึกลง Database

จากการทดสอบพบว่าในตาราง `deposit_transaction` มีข้อมูล 2 รายการ ดังนั้นรายการที่สองยังถูกบันทึก แม้ Request จะเกิด Error ก็ตาม

14. คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

@Transactional ใช้กำหนดให้คำสั่งภายใน Method ทำงานอยู่ภายใต้ Transaction เดียวกัน หากทำงานสำเร็จทั้งหมดจะทำการ COMMIT แต่ถ้าเกิด Error จะทำการ ROLLBACK ข้อมูลกลับไปก่อนเริ่ม Transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงินมีการทำงานมากกว่าหนึ่งขั้นตอน ได้แก่

- เพิ่มยอดเงินใน Account
- บันทึกประวัติการฝากเงิน

ทั้งสองขั้นตอนควรสำเร็จพร้อมกัน หากขั้นตอนหนึ่งเกิด Error อีกขั้นตอนก็ควรถูกยกเลิกด้วย เพื่อป้องกันข้อมูลใน Database ไม่สอดคล้องกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันและบันทึกการเปลี่ยนแปลงลง Database อย่างถาวร

ROLLBACK คือการยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นใน Transaction และย้อนกลับไปยังสถานะก่อนเริ่ม Transaction

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ One-to-Many คือ Account 1 บัญชี สามารถมี DepositTransaction ได้หลายรายการ ดังนั้น DepositTransaction จึงมีความสัมพันธ์แบบ @ManyToOne กลับไปยัง Account

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลที่เกิดขึ้นภายใน Transaction จะถูก ROLLBACK ทั้งหมด จากการทดลองใน Lab เมื่อฝากเงิน 1000 บาท แล้วเกิด Test Rollback

- ยอด Account ยังคง 1000 บาท
- ประวัติการฝากยังมีเพียง 1 รายการ
- รายการที่กำลังทำอยู่ถูกยกเลิก

ดังนั้น Database จะไม่เก็บข้อมูลที่เกิดขึ้นจาก Transaction ที่ล้มเหลว

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อไม่มี @Transactional แล้วเกิด Error ระบบจะไม่สามารถย้อนกลับการเปลี่ยนแปลงทั้งหมดได้เหมือนเดิม ทำให้ข้อมูลที่บันทึกสำเร็จก่อนเกิด Error ยังคงอยู่ใน Database

จากการทดสอบพบว่า deposit_transaction มี 2 รายการ แม้ Request ครั้งที่สองจะตอบ 500 Internal Server Error

มี @Transactional → Error → ROLLBACK ทั้งหมด

ไม่มี @Transactional → Error → ข้อมูลบางส่วนยังถูกบันทึก